Decomposing a factorial into large factors

Terence Tao · 26 March 2025

https://terrytao.wordpress.com/2025/03/26/decomposing-a-factorial-into-large-factors/

tao2typ · converted to Typst 2026-10-14 (UTC)

I've just uploaded to the arXiv the paper "Decomposing a factorial into large factors". This paper studies the quantity $t(N)$, defined as the largest quantity such that it is possible to factorize $N!$ into N factors $a_1, ..., a_N$, each of which is at least $t(N)$. The first few values of this sequence are

$$1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, ...$$

(OEIS A034258). For instance, we have $t(9) = 3$, because on the one hand we can factor

$$9! = 3 \times 3 \times 3 \times 3 \times 4 \times 4 \times 5 \times 7 \times 8$$

but on the other hand it is not possible to factorize $9!$ into nine factors, each of which is 4 or higher.

This quantity $t(N)$ was introduced by Erdös, who asked for upper and lower bounds on $t(N)$; informally, this asks how equitably one can split up $N!$ into N factors. When factoring an arbitrary number, this is essentially a variant of the notorious knapsack problem (after taking logarithms), but one can hope that the specific structure of the factorial $N!$ can make this particular knapsack-type problem more tractable. Since

$$N! = a_1 ... a_N \geq t(N)^N$$

for any putative factorization, we obtain an upper bound

$$t(N) \leq (N!)^{1/N} = \frac{N}{e} + O(\log N) \tag{1}$$

thanks to the Stirling approximation. At one point, Erdös, Selfridge, and Straus claimed that this upper bound was asymptotically sharp, in the sense that

$$t(N) = \frac{N}{e} + o(N) \tag{2}$$

as $N \to \infty$; informally, this means we can split $N!$ into N factors that are (mostly) approximately the same size, when N is large. However, as reported in this later paper, Erdös "believed that Straus had written up our proof... Unfortunately Straus suddenly died and no trace was ever found of his notes. Furthermore, we never could reconstruct our proof, so our assertion now can be called only a conjecture".

Some further exploration of $t(N)$ was conducted by Guy and Selfridge. There is a simple construction that gives the lower bound

$$t(N) \geq \frac{3}{16} N - o(N)$$

that comes from starting with the standard factorization $N! = 1 \times 2 \times ... \times N$ and transferring some powers of 2 from the later part of the sequence to the earlier part to rebalance the terms somewhat. More precisely, if one removes one power of two from the even numbers between $\frac{3}{8}N$ and N, and one additional power of two from the multiples of four between $\frac{3}{4}N$ to N, this frees up $\frac{3}{8}N + o(N)$ powers of two that one can then distribute amongst the numbers up to $\frac{3}{16}N$ to bring them all up to at least $\frac{3}{16}N - o(N)$ in size. A more complicated procedure involving transferring both powers of 2 and 3 then gives the improvement $t(N) \geq \frac{1}{4}N - o(N)$. At this point, however, things got more complicated, and the following conjectures were made by Guy and Selfridge:

- (i) Is $\frac{t(N)}{N} \leq \frac{1}{e}$ for all $N \neq 1, 2, 4$?
- (ii) Is $t(N) \geq \lfloor 2N/7 \rfloor$ for all $N \neq 56$? (At $N = 56$, this conjecture barely fails: $t(56) = 15 < 16 = \lfloor 2 \times 56/7 \rfloor$.)

- (iii) Is $\frac{t(N)}{N} \geq \frac{1}{3}$ for all $N \geq 300000$?

In this note we establish the bounds $$\frac{1}{e} - \frac{O(1)}{\log N} \leq \frac{t(N)}{N} \leq \frac{1}{e} - \frac{c_0 + o(1)}{\log N} \quad (3)$$ as $N \to \infty$, where c_0 is the explicit constant

$$c_0 := \frac{1}{e} \int_0^1 \left\lfloor \frac{1}{x} \right\rfloor \log\left(ex \left\lceil \frac{1}{ex} \right\rceil \right) dx \approx 0.3044.$$

In particular this recovers the lost result (2). An upper bound of the shape

$$\frac{t(N)}{N} \leq \frac{1}{e} - \frac{c + o(1)}{\log N} \quad (4)$$

for some $c > 0$ was previously conjectured by Erdös and Graham (Erdös problem #391). We conjecture that the upper bound in (3) is sharp, thus

$$\frac{t(N)}{N} = \frac{1}{e} - \frac{c_0 + o(1)}{\log N}, \quad (5)$$

which is consistent with the above conjectures (i), (ii), (iii) of Guy and Selfridge, although numerically the convergence is somewhat slow.

The upper bound argument for (3) is simple enough that it could also be modified to establish the first conjecture (i) of Guy and Selfridge; in principle, (ii) and (iii) are now also reducible to a finite computation, but unfortunately the implied constants in the lower bound of (3) are too weak to make this directly feasible. However, it may be possible to now crowdsource the verification of (ii) and (iii) by supplying a suitable set of factorizations to cover medium sized N, combined with some effective version of the lower bound argument that can establish $\frac{t(N)}{N} \geq \frac{1}{3}$ for all N past a certain threshold. The value $N = 300000$ singled out by Guy and Selfridge appears to be quite a suitable test case: the constructions I tried fell just a little short of the conjectured threshold of 100000, but it seems barely within reach that a sufficiently efficient rearrangement of factors can work here.

We now describe the proof of the upper and lower bound in (3). To improve upon the trivial upper bound (1), one can use the large prime factors of $N!$. Indeed, every prime p between N/e and N divides $N!$ at least once (and the ones between N/e and $N/2$ divide it twice), and any factor a_i that contains such a factor therefore has to be significantly larger than the benchmark value of N/e. This observation already readily leads to some upper bound of the shape (4) for some $c > 0$; if one also uses the primes p that are slightly less than N/e (noting that any multiple of p that exceeds N/e, must in fact exceed $\lceil N/ep \rceil p$) is what leads to the precise constant c_0.

For previous lower bound constructions, one started with the initial factorization $N! = 1 \times \ldots \times N$ and then tried to "improve" this factorization by moving around some of the prime factors. For the lower bound in (3), we start instead with an approximate factorization roughly of the shape

$$N! \approx (\prod_{t \leq n < t + 2N/A,\ \text{odd}} n)^A$$

where t is the target lower bound (so, slightly smaller than N/e), and A is a moderately sized natural number parameter (we will take $A \asymp \log^3 N$, although there is significant flexibility here). If we denote the right-hand side here by B, then B is basically a product of N numbers of size at least t. It is not literally equal to $N!$; however, an easy application of Legendre's formula shows that for odd small primes p, $N!$ and B have almost exactly the same number of factors of p. On the other hand, as B is odd, B contains no factors of 2, while $N!$ contains about N such factors. The prime factorizations of B and $N!$ differ somewhat at large primes, but B has slightly more such prime factors as $N!$ (about $\frac{N}{\log N} \log 2$ such factors, in fact). By some careful applications of the prime number theorem, one can tweak some of the large primes appearing in B to make the prime factorization of B and $N!$ agree almost exactly, except that B is missing most of the powers of 2 in $N!$, while having some additional large prime factors

beyond those contained in $N!$ to compensate. With a suitable choice of threshold t, one can then replace these excess large prime factors with powers of two to obtain a factorization of $N!$ into N terms that are all at least t, giving the lower bound.

The general approach of first locating some approximate factorization of $N!$ (where the approximation is in the "adelic" sense of having not just approximately the right magnitude, but also approximately the right number of factors of p for various primes p), and then moving factors around to get an exact factorization of $N!$, looks promising for also resolving the conjectures (ii), (iii) mentioned above. For instance, I was numerically able to verify that $t(300000) \geq 90000$ by the following procedure:

- Start with the approximate factorization of $N!$, $N = 300000$ by $B = (\prod_{90000 \leq n < 102000,\ \mathrm{odd}} n)^{50}$. Thus B is the product of N odd numbers, each of which is at least 90000.
- Call an odd prime B-heavy if it divides B more often than $N!$, and $N!$-heavy if it divides $N!$ more often than B. It turns out that there are 14891 more B-heavy primes than $N!$-heavy primes (counting multiplicity). On the other hand, $N!$ contains 2999992 powers of 2, while B has none. This represents the (multi-)set of primes one has to redistribute in order to convert a factorization of B to a factorization of $N!$.
- Using a greedy algorithm, one can match a B-heavy prime p' to each $N!$-heavy prime p (counting multiplicity) in such a way that $p' \leq 2^{m_p} p$ for a small m_p (in most cases one can make $m_p = 0$, and often one also has $p' = p$). If we then replace p' in the factorization of B by $2^{m_p} p$ for each $N!$-heavy prime p, this increases B (and does not decrease any of the N factors of B), while eliminating all the $N!$-heavy primes. With a somewhat crude matching algorithm, I was able to do this using $\sum_p m_p = 39992$ of the 299992 powers of 2 dividing $N!$, leaving 260000 powers remaining at my disposal. (I don't claim that this is the most efficient matching, in terms of powers of two required, but it sufficed.)
- There are still 14891 B-heavy primes left over in the factorization of (the modified version of) B. Replacing each of these primes with $2^{17} \geq 90000$, and then distributing the remaining $260000 - 17 \times 14891 = 6853$ powers of two arbitrarily, this obtains a factorization of $N!$ into N terms, each of which are at least 90000.

However, I was not able to adjust parameters to reach $t(300000) \geq 100000$ in this manner. Perhaps some readers here who are adept with computers can come up with a more efficient construction to get closer to this bound? If one can find a way to reach this bound, most likely it can be adapted to then resolve conjectures (ii) and (iii) above after some additional numerical effort.

UPDATE: There is now an active Github project to track the latest progress, coming from multiple contributors.

Comments (83)

Anonymous · 26 March 2025

there is an open brace for href
[Corrected, thanks – T.]

Samuel Bonaya Buya · 26 March 2025

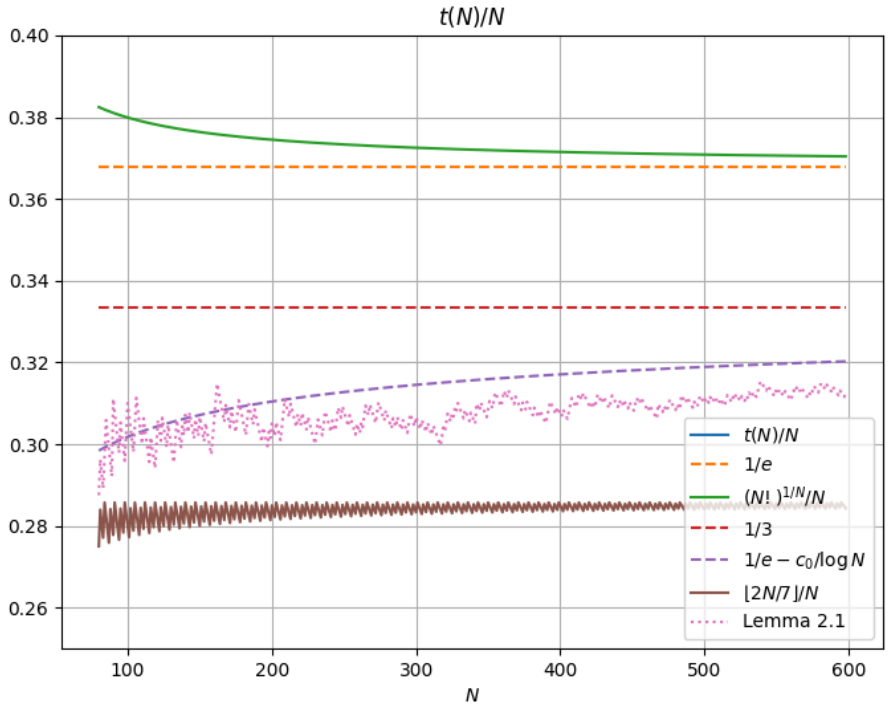
In my opinion the paper is a significant contribution by Tao on the understanding of the prime number theorem

> **Anonymous** · 12 April 2025
>
> No it's not

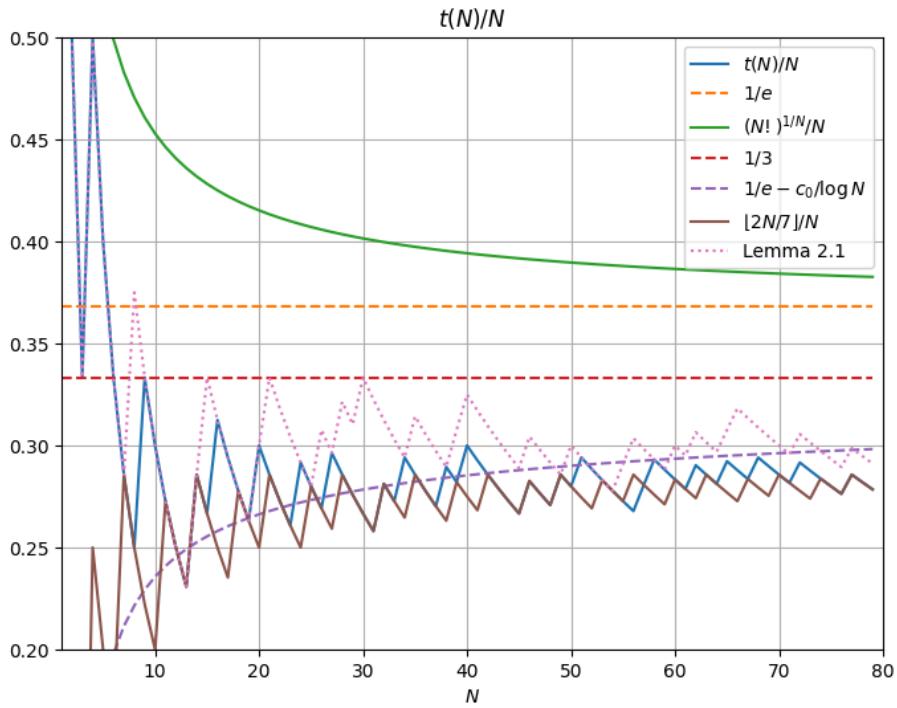
Antoine Deleforge · 27 March 2025

Looking at the first few numbers in the OEIS sequence, it looks like t(n+1) – t(n) is always zero or one. Is there any reason for this to be true?

> **Terence Tao** · 27 March 2025
>
> No; in fact, in Guy's article on this problem, he notes that there is a jump of 2 from $t(124) = 35$ to $t(125) = 37$ (though he does not provide enough preceding values to extend the sequence in the OEIS). In that article he also notes that Erdos conjectures that the gaps can in fact be arbitrarily large, though I see no way to attack this question even heuristically (as the extremizers for this problem may be neither structured nor (pseudo)random, but exhibit some very strange intermediate behavior).
> In the image below, I display the upper bound on $t(N)/N$ (the pink dots) in the intermediate range $80 \leq N \leq 599$ coming from Lemma 2.1 of my paper (there is no plot for $t(N)/N$ in this image as I do not have data in this range). [Incidentally there is a slight typo in that lemma, which I will correct in the next revision: the term $\sqrt{t}+1$ should instead be $\lfloor \sqrt{t} \rfloor$.] There is considerable fluctuation here (due to the corresponding fluctuation in the primes), which is also reflected in the related plot in Figure 2 of the paper. Of course, fluctuation in the upper bound for $t(N)/N$ does not imply fluctuation in the true value of $t(N)/N$, but it is perhaps evidence in that direction.



And below is a comparison of the upper bound against the true value of $t(N)/N$ in the range $N \leq 79$:



Ivan · 27 March 2025

You may want to enclose the comma in curly brackets when it is used as a thousands separator so that $LaTeX$ does not generate extra space after it, e.g., $300,000$ instead of $300, 000$ (p. 3 of the paper).

[Thanks, this will be done in the next revision of the ms -T]

Antoine Deleforge · 27 March 2025

I don't immediately see the connection to the knapsack problem. If we pick the non-dividable items to be the logs of the prime factors of N!, then the problem amounts to distributing *all* of these items into N knapsacks, such that each knapsack contains *at least* a value of t(N). This is quite different from the original knapsack problem where the goal is rather to select a *subset* of items, and maximize the value while remaining *below* the knapsack capacity. Is there a deeper or more natural connection that I am missing? Can further progress on this Erdos problem be expected to eventually yield insights on the knapsack problem, or is the relation between the two too distant for that to happen?

> **Terence Tao** · 27 March 2025
>
> It's more accurate to say that the factorial problem is a *variant* of the knapsack problem; most directly, it corresponds to a knapsack problem with negative item sizes (and negative capacity in the backpack), which of course is not physically realistic (or intuitive), but it is possible that some of the knapsack algorithms that work for positive sizes and capacities can carry over to this new context with suitable modification. (For instance, I would guess that the problem of solving this sort of factoring problem for a general input number (rather than a factorial) is NP-complete, by some modification of the proof of NP-completeness of the knapsack problem.)
> Note by the way that to solve the factorization problem, it suffices to distribute some subset of the log-primes into the knapsacks rather than all of them, since one can just add in the remaining log-primes arbitrarily to finish the job.
>
> > **Anonymous** · 28 March 2025
> >
> > At first view it looks that this is closer to a Bin packing problem than to a knapsack problem.
> >
> > > **Terence Tao** · 29 March 2025
> > >
> > > Following your hint, it seems in fact that this problem is a special case of the bin covering problem, which is dual to the bin packing problem.

ducduc2710 · 27 March 2025

That technique I think can be used to limit prime gaps.

Anonymous · 27 March 2025

Small typo: "multiples of four between 3/4 to N"
It should be 3/4N to N.
[Corrected, thanks -T.]

Terence Tao · 27 March 2025

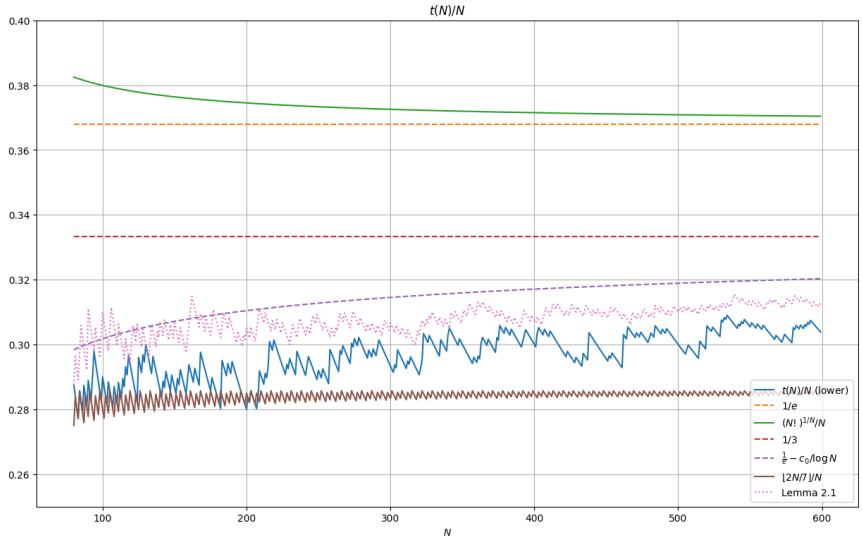
I'm posting (with permission) some computational work by Andrew Sutherland, who implemented a greedy approach working through the prime factors p of $N!$ in
reverse order (with multiplicity), constucting integers of the form $cp \geq t$ with c chosen to be minimal subject to the constraint that it can be constructed from the divisors of $N!$ that still remain. For instance when $N = 300000$, it is able to factor $N!$ into 300372 numbers greater than or equal to 100000, verifying the Guy-Selfridge conjecture at this value. The code (in Maple) is at https://math.mit.edu/~drew/GuySelfridge.m . An earlier (less efficient) factorization with these parameters can be found at https://math.mit.edu/~drew/ES300000.txt .

Andrew writes, " It only takes about a few seconds on a fast machine so I was able to run it on all N in $[100000, 500000]$ and noticed that while it typically succeeds on $N \geq 300000$, it still fails to prove $t(N) \geq N/3$ in 922 cases in $[300000, 500000]$, including N as large as 490230. But it succeeds on every N in $[298344, 300000]$, so if the conjecture is true, it is still true if you replace 300000 with 298344 (probably this can be lowered a lot further, the greedy approach is not optimal).
I then tested the threshold $\lfloor 2N/7 \rfloor$ on all N from 5 to 100000. It failed only for $N = 56, 182, 200, 207$".
With this data, one can now reduce conjecture (ii) to conjecture (iii) provided one can construct suitable factorizations of $N!$ to resolve the three remaining cases $N = 182, 200, 207$ (one also has to retest the range $100000 < N \leq 300000$ but this should be straightforward, since there is now enough room that one should be able to sample this range quite sparsely, e.g., test the threshold $\lfloor 2N/7 \rfloor + 2000$ for N a multiple of 7000). But the range $N \geq 300000$ seems a bit more delicate, as the most direct greedy algorithm sometimes fails.
Andrew adds, "I think it's possible that one might be able to turn this algorithm into an asymptotic bound. For sufficiently large prime divisors p of $N!$, you can just take $c = \lceil T/p \rceil$ because c will be small and there are plenty of powers of small primes initially available (and you can quantify this), and even if you focus just on the integers the algorithm constructs before it hits the first cofactor it cannot minimize you should get some constant factor of N that might be bigger than $1/4$."



Terence Tao · 27 March 2025

Andrew has kindly shared with me the lower bounds for $t(N)$ for $80 \leq N \leq 600$ given by this approach, and I have incorporated them into my previous plot here, showing the current upper (pink) and lower (blue) bounds on $t(N)/N$:



Note the verification of the conjecture $t(N) \geq \lfloor 2N/7 \rfloor$ in this range for $N \neq 182, 200, 207$. The data also replicates Guy's reported values $t(124) = 35, t(125) = 37$, and suggests that the jumps in t are indeed rather irregular. (The text file for the data can be found here.)



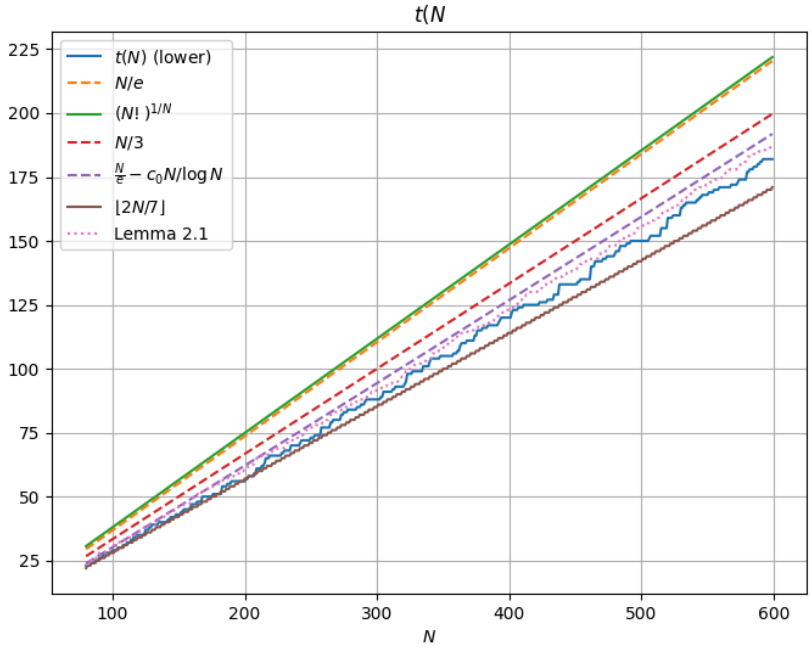
Anonymous · 31 March 2025

shouldn't lower bound also be a weakly increasing function?
...I guess that's a post-processing step



Terence Tao · 31 March 2025

$t(N)$ is a weakly increasing function, but the ratio $t(N)/N$ can oscillate. Here is a plot of the unnormalized bounds:





frobitzblog · 27 March 2025

With a bit of fiddling by hand I was able to improve the factorizations for N=182,200,207, so now (ii) is confirmed up to 100,000. You can find the factorizations here, here, and here.



hideoutinstantab46bc0124 · 20 April 2025

I personnally link this to Erdös-Tenenbaum's formula

$$\mathbb{E}[p_{\min}(n)] = \frac{e^{-\gamma}\log x}{\log\log x + M - \frac{\gamma}{2}} + O\left(\frac{\log x}{(\log\log x)^2}\right)$$



Anonymous · 28 March 2025

Should inequality (4) have t(N)/N on the left-hand side (as opposed to just t(N))?
[Corrected, thanks – T.]



Terence Tao · 28 March 2025

A back of the envelope calculation suggests that the upper bound in my paper, if made explicit, would verify the conjecture $t(N)/N \geq 1/3$ for roughly $N \geq 10^{11}$. (There are particular inefficiencies when $N/3$ is slightly larger than a power of two, as it then becomes expensive to adopt strategies that save the powers of two for last.) So some additional constructions will be needed to cover the medium range $3 \times 10^5 \leq N \leq 10^{11}$.

A natural idea to improve both the numerics and the asymptotics is to mix and match powers of 2 and 3 in the endgame when these are the only primes left to assign, taking advantage of the incommensurability of $\log 2$ and $\log 3$ to get more accurate matches to a target threshold. This does make the algorithms and analysis more complicated, though.



Terence Tao · 29 March 2025

After running Andrew's code to verify $t(300000) \geq 100000$ I noticed that the greedy algorithm exhausts the factors of 2 and 3 relatively early; by the time the algorithm reaches the prime 29, the factors of 2 and 3 are already gone and so one has to use primes 5 and higher to fill up the remaining factors. This is likely the main source of inefficiency in the direct greedy algorithm. Some sort of ad hoc modification of the greedy algorithm in which some preference is given to terms that avoid 2 and 3 may improve performance. If one can get to a final stage where only powers of 2 and 3 remain, then it may also make sense to switch from a greedy method to a linear programming method: specifically, if one wants to distribute $2^a 3^b$, one can locate the smallest $2^{a_1} 3^{b_1} \geq t$ with $a_1/b_1 \geq a/b$, and the smallest $2^{a_2} 3^{b_2} \geq t$ with $a_2/b_2 \leq a/b$, and express (a, b) as an integer linear combination of (a_1, b_1) and (a_2, b_2) (plus a negligible error which one may simply discard) to get a pretty good factorization.



Terence Tao · 29 March 2025

Currently, Andrew's code permits one to factor $N!$ for $N = 300000$ into $N + 372$ factors greater than or equal to $N/3$, so one has a surplus of 372 factors here. I am hoping that this surplus can be increased somewhat through some tweaking of the greedy algorithm, which should allow one to cover more ranges for the conjecture $t(N) \geq N/3$. I have in mind something like this:

1. First, apply the greedy algorithm to remove all primes larger than, say, $N/30$. These primes will be problematic no matter what algorithm one chooses (it is rare for their multiples to be very close to N) and so one may as well dispose of them as efficiently as possible immediately.
2. Now, take all the numbers between $N/3$ and (say) $N/3 + 3000$ that are coprime to both 2 and 3, and remove them from the remaining portion of $N!$ (only if they are available, of course). Iterate this greedily until no further such numbers can be removed. One should now be left with mostly copies of 2 and 3 and only a small number of remaining primes to allocate (the point being that the numbers between $N/3$ and $N/3 + 3000$ have a very similar distribution of prime factors to the numbers between 1 and N, except at the large primes which we have just removed).
3. Use the greedy algorithm again to eliminate all primes larger than 3. This should leave a large pool of copies of 2 and 3.
4. Use the linear programming method in the previous comment to group these copies of 2 and 3 into quantities $2^a 3^b$ slightly larger than $N/3$.
5. Apply the greedy algorithm to clean up any leftovers.

Unfortunately I will not have time today to try to implement such a scheme, but perhaps other commenters could try this (or some other method) to improve upon the previous surplus of 372, which I think is a reasonable proxy for algorithmic efficiency. Note for this value of N that $\log N!/\log(N/3) \approx N + 2570$, which provides a hard upper limit for the surplus (and due to the large primes which can't be multiplied to be close to $N/3$, the hard limit is actually a little less than this). Still, there is room for improvement beyond 372.

EDIT: by taking into account the amount by which all large primes will go over $N/3$ (see Lemma 2.1 of the paper; the gap between the LHS and RHS of (2.1), divided by $\log t$, upper bounds the surplus), I now revise the theoretical upper limit for the surplus to just 454. So not as much room for improvement as I thought… the greedy algorithm is actually quite efficient! This also suggests that if the greedy algorithm fails significantly at some N slightly larger than 300000, then the Guy-Selfridge conjecture may in fact be false.



Terence Tao · 29 March 2025

I was able to get the surplus in the $t(300000) \geq 100000$ verification up to 410 (here is the list of factors) by an extremely ad hoc method; I had attempted to coax the greedy algorithm to set aside some powers of 2 and 3 for later use, which helped somewhat (I could improve 372 to 401 this way), but then by accidentally putting in some rather bad choices of parameters (which used up the powers of 2 and 3 prematurely), I mysteriously got an improvement to 410. It seems there are a lot of discontinuities in this problem; I am not sure how to systematize the search for better constructions.

Anonymous · 29 March 2025

@**Ivan** Or even better; use num from the siunitx package (or qty for physical quantities) when typing numbers with more than four digits on either side of the decimal marker.

Anonymous · 29 March 2025

I've only been experimenting with small factorials, but fractional improvements can be made by optimising the factorisation process.

Anonymous · 29 March 2025

Sorry, that last post was mine.
I spent the day writing a php program to check t(N). The beauty is I don't need to work out the entire factorial, only the products of two of its factors. It was a bit involved and has a few shortcomings, but it gets the job done. It flies through the lower numbers but struggles with higher T's. I can see the primes appearing as the array iterates. I'm just an amateur, but it was fun to see the final result.
For example:
t(100): 29 = 3.448 (Script Execution Time = 0.426 sec).
t(300): 73 = 4.109 (58.697 sec).
t(600): 148 = 4.054 (3 min 46.433 sec).
I appreciate the skill of mathematicians like you who are able to improve t(n) with greater precision.
Alan



jsahil730 · 2 April 2025

Is it possible to share the source code?

Anonymous · 1 April 2025

I've been thinking about the problem, and realise you need to break the factors down into its own factors to tackle the problem. I'll use t(100) as an example. Ideally, if we weren't dealing with whole numbers, the precise result would be 100! ° 1/100 = 37.992. 100/e = 36.787, which is very close. Interestingly, the number of identical products of two factors is 1737/4950 = 35.09%. My result of 29 is average and was based on the original factors, but I'll explore one level deeper.
Alan

Anonymous · 1 April 2025

the Spanner theorem : Distribution of the Composite numbers as Complement of the Primes in the set {1,2,3,4 ...}
Jenda Vondra 2024
do you want it folks? .. just ask me nicely
gives the EXACT factorization of every composite number up to any number n .. each of these numbers is of course composed of the primes up to n/2 .. simply Eratothsenes seive(but only doing crossings out ONCE) .. computer does that for u .. primes are those numbers left uncrossed .. perfectly

countable too .. since we count the composite numbers as we make them .. in sequential order of course
now of course to make these primes up to n .. we need to know the primes up n/2 .. seems hard .. but it's not .. to get the primes up to n/2 .. we just use the same process as we used to get them up to n .. but start with knowing the primes up to n/4
should be getting obvious now .. that to get the primes up to n/4 .. we only need to know the primes up to n/8
once more .. to get the primes up to n/8 .. what do we need?
yes .. we only need the primes up to.. u guessed it .. n/16
the computer time is halved each time too .. lucky .. next list of primes we need is only up to n/32 to get them to n/16
so it goes on .. pretty soon we get back to the first prime .. number 2
so let's go the other way .. use the multiplicative operation .. in order
the prime 2 .. gives the primes up to 4 .. namely 3
primes 2 and 3 by multiplicative operation(in orderly fashion) give the primes up to 8
primes 2 3 5 7 then give(same orderly multiplication with ONE crossing out each time)(remember the computer is recording the exact prime factorization by virtue of MAKING each composite number and writing down what it is)
so we get the primes 2 3 5 7 11 13 (n=16)
next lot written out is 2 3 5 7 11 13 17 19 23 29 31 (n=32)
and so on
In the case of some factorization of n! .. it's a trivial quest
rhe Spanner theorem writes out each composite number prime factorization .. up to any number n! = N in polynomial time O(N) with the exponent on N being just less than 1 as N gets large



Anonymous · 1 April 2025

What do the + signs in subscripts mean in Proposition 3.1 in the paper? I cannot find this notation defined in the paper.
[$x_+ := \max(x, 0)$ denotes the positive part of x. -T]



Matthieu Rosenfeld · 3 April 2025

I have written a heuristic that seems to work to improve the factorization compared to the greedy factorization. Whenever you use the largest remaining prime p you want to construct a factorization of i=n/(3p) if possible (otherwise, you try to construct i+1, i+2...). Suppose the factorization of i+1 uses less factors than the factorization of i then it could be a better choice to construct i+1 instead as we keep more smaller factors for the rest of the packing procedure.
Here is a link to a c++ code doing just that:
https://www.lirmm.fr/~mrosenfeld/things/test_v2.cpp
The interesting part are the 3 lines 132-135.
For 300000, it gives a surplus of 393, which i can then slightly improve to a surplus of 415 by adding 62 occurrences of 2*2*2*2*2*5*5*5*5*5 (this part is completely ad-hoc) [https://www.lirmm.fr/~mrosenfeld/things/decomposition300000.out]
For 490230, the same code (without the add-hoc trick), yields a surplus of 553 instead of having not enough factors [https://www.lirmm.fr/~mrosenfeld/things/decomposition490230.out].



Anonymous · 3 April 2025

The process of increasing t(N) is complicated in that there are two ways to simplify: break down the number into two factors, or peform swaps of products. 9 can be broken down into 3×3, but 9×2

= 6*3 (9->6 and 2->3). They both increase t(9) to 3 but higher numbers would have to involve both processes in some ratio.
Alan

 **uhrmar** · 3 April 2025

I might be late to the party, but I was able to make the approach of Andrew Sutherland non-greedy using linear programming. Surprisingly, the solutions are 'almost integral' and quite good by just rounding down the fractional components. The residual factor (after dividing out the integral LP solution) can then be further processed using the greedy strategy.
This method finds for the benchmark problem $N = 300000$, $t(N) = 100000$ a solution with $N + 445$ factors (which should be optimal, since the LP upper bound is 300445.8339878815.
I've published the code in this repository: https://gitlab.com/uhrm/facfac

 **Terence Tao** · 3 April 2025

Thanks to both you and Matthieu for these (surprisingly efficient) numerical explorations! It now seems hopeful that we can verify the Guy-Selfridge conjecture at the scale of $N = 300000$ (and I now suspect that the authors had some inkling that this was close to the threshold, given how narrow the margin of error is here).
Because $(N/3)^{N+s} \approx (N/3 + s\log(N/3)/3)^N$, I would expect heuristically that a surplus of s should allow one to raise the lower bound on the factors from $N/3$ to about $N/3 + s\log(N/3)/3$, and thus also automatically verify the conjecture for the next $s\log(N/3)$ or so elements past N from the monotone nature of $t(N)$. For instance for $N = 300000$ and $s = 445$ this predicts the ability to cover a range of about 5000 or so. So one might estimate that to get from this up to say $N = 10^6$ one might need to perform one of these algorithms a couple hundred times, which seems quite feasible. As N increases, I expect the available surplus to also increase at an eventually linear fashion (asymptotically it is $N(1/e - 1/3 - o(1))$, with the error decaying at a logarithmic rate), so hopefully one can scale up to $N = 10^7$, $N = 10^8$, etc. at a subexponential rate. As I said before, my own theoretical arguments seem to kick in at about 10^{11}, which may be slightly out of reach of brute forcing the remaining ranges, but perhaps these sorts of optimizations may also suggest a new construction to handle very large N that one could also try to analyze theoretically. (I could imagine there being a speed/simplicity vs. strength tradeoff; there may be constructions that offer a small surplus, but are faster to compute numerically and also easier to analyze theoretically. So as N increases and there is more surplus to play with one might transition into simpler and faster constructions.)
p.s. I have decided not to submit my paper for now in view of the ongoing numerical work.
If it looks like we have a real shot at the Guy-Selfridge conjecture, then I would be happy to convert the paper into a collaborative project that incorporates both the theoretical analysis and numerical validation.
p.p.s. if the linear program provides a rigorous upper bound on $t(N)$ then it may now be possible to extend OEIS A034258 if the bound is tight enough to match the known constructions.

 **ventullo** · 5 April 2025

Hi,
I took the ideas from this comment to validate $t(N) >= N/3$ for $N \in [3*10^5, 10^8]$. It took about 7h22m in total. By the end, it took about 5 minutes to greedily compute a factorization for N ~ 10^8.
I used my own implementation of Sutherland's greedy algorithm, together with a heuristic based on the comment. Instead of calculating surpluses, I set the target lower bound to be a little higher than $N/3$, something like $N*(\frac{1}{3} + \epsilon)$. Finding a solution thus validates the entire

range $[N, (1+3\epsilon)N]$. As for the choice of epsilon, I hardcoded it at .01 to start, bumped it up to .02 once $N > 3*10^6$, then bumped it to .03 once $N > 3*10^7$.
Occasionally the greedy algorithm fails, but that's okay: just decrement N and try again. It usually doesn't take too many retries to work. That said, I noticed the existence of "challenging ranges", where many nearby N's resist the greedy approach. So, I did what I believe is a slight optimization, where instead of decrementing N, I subtract .00001*N rounded. Failures became very rare in the higher ranges, suggesting I could probably bump up that coefficient some more.
Code is here: https://github.com/kevinventullo/TaoFactorial/



Terence Tao · 5 April 2025

That's great progress! It suggests that we could get another order of magnitude or so by just being much more efficient in implementing the vanilla greedy algorithm. One thing I noticed when running the greedy algorithm myself was that it spent the bulk of its time at the end of the process when only small primes like 2, 3, and 5 remain, and the factors acquire a high multiplicity. One possible speedup is, when removing a factor from the remaining portion of the factorial, to automatically remove the maximum number of copies of that factor from the factorial rather than have a loop that removes them one by one. For instance, if one has located a factor $2^a 3^b 5^c$ and the remaining expression to be factorized is $2^A 3^B 5^C \dots$, then pull out $\lfloor \min(A/a, B/b, C/c) \rfloor$ copies of $2^a 3^b 5^c$ right away. This may give some speedup. (Also, for the purposes of storing the factorization for future verification, it is probably more compact to store the factors with multiplicity, which this procedure does automatically.)
Another thing that can help is that the initial stages of the greedy algorithm are quite predictable and could be largely precomputed, in a way that dispenses with the need to store information about large primes. For instance, the primes greater than or equal to t simply remove themselves from the factorial without any further impact (other than each prime contributing one factor to the factorization) so one could already start by restricting attention to those primes less than t. More generally, if one runs the algorithm to exhaust down to prime factors $p \geq t/K$ for some natural number K that is not too large (in particular $K \leq \sqrt{t}$), then what should happen is that all the primes appearing in $N!$ in the range $t/k \leq p < t/k - 1$ for some $2 \leq k \leq K$ will pull out a factor of k from the factorial when the greedy algorithm is run (and as stated before, the primes above t will not pull out any factor at all). So, the net effect of running the greedy algorithm down to the threshold t/K is that one keeps all the primes in $N!$ below t/K, with an additional $M_k := \sum_{t/k \leq p < t/k-1} \left\lfloor \frac{N}{p} \right\rfloor$ factors of k removed for each $2 \leq k \leq K$, and one has already acquired $M_1 + \dots + M_K$ factors. So if one has a rapid way to compute M_k (or even just to get an accurate upper bound) for $2 \leq k \leq K$, one can initialize the greedy algorithm only using primes up to t/K, which may save some run time as well as memory. In principle one may be able to take K all the way up to $\sqrt{t}$ without running out of primes, thus theoretically gaining something like a square root speedup in run time and memory usage.
There is a formula for M_k in terms of the prime counting function π:

$$M_k = \sum_{j \geq 1} j \sum_{t/k \leq p < t/k-1; N/(j+1) < p \leq N/j} 1$$
$$= \sum_{j : a_{j,k} \leq a_{j+1,k}} j(\pi(a_{j,k}) - \pi(a_{j+1,k}))$$

where $a_{j,k} := \min(\lfloor N/j \rfloor, \lceil t/k - 1 \rceil - 1)$. So, with precomputed values of the prime counting function, or with accurate upper bounds from effective versions of the prime

number theorem, one should be able to compute M_k relatively quickly, allowing one to reduce the problem to just the primes up to t/K for fairly large values of K.

 **Matthieu Rosenfeld** · 6 April 2025

I was also trying to use the fact that we have some margin to improve the efficiency of the verification. In particular, I spent a lost of time improving my code by using the fact that most of the time one easily build a factorization for (3(k+1))! from the factorization of (3k)!. It works reasonably well, and I was able to reach 20.000.000 in 20 minutes (and I did not run it for more than this).
But it seems much better to do the range as suggested by Ventullo. My current implementation [here] of this idea takes 13 minutes to go from 300.000 to 10^8 and 41 minutes to go to 3*10^8. At this point, it slows down because it uses almost all my RAM, so I will need to wait Monday to run it for 10^9 or more on a machine with more RAM .
A few implementations details:

- I am using a parameter epsilon that I decrease ("step by step" from 62 to 0 if it comes to this) until I find a factorization with n factors of size (n*(1000+eps)/1000)/3.
- I am still using the heuristic that I previously mentioned (in the greedy algorithm, we sometimes choose a larger factor if it is the product of fewer small primes).
- If a factor can be added multiple time, then I add as many copies of it as possible at once (I was already doing this).
- One should stop the factorization process as soon as we have found the desired amount of large enough factors. It seems to save more time than what I would expect.
- Note that I pre-compute the factorization of every integers (storing for each integer the smallest prime number that divides it) and the list of prime numbers. This would not be much more expensive to also precompute the prime counting functions, although I am not sure that It would save a lot (as you mentioned it seems most of the running time is already spent on the smallest factors). I will try to implement this tomorrow to see how much this helps.

(I suspect that a part of the speed difference might also be due to me using C++ instead of Python).
On the other side, adding a second magic heuristic parameter allows getting down to 80000. But I think this heuristic approach will fail between 60000 and 80000 (as it currently fails for many of the values in between).

 **Terence Tao** · 6 April 2025

Thanks for the new code runs! If I now understand correctly, the conjecture $t(N) \geq N/3$ is now verified in the range $8 \times 10^4 \leq N \leq 3 \times 10^8$. Combined with Andrew's verification of $t(N) \geq \lfloor 2N/7 \rfloor$ for $N \leq 10^5$, $N \neq 57$ (which presumably can also be replicated by the newer code), we have now reduced the second Guy-Selfridge conjecture to the third (and probably there is already enough room for the analytic approach to fully resolve the second conjecture in the remaining range $N \geq 3 \times 10^8$, though this would be redundant if we can settle the third in this range).
Do you have a sense of how much surplus there is for say $N = 3 \times 10^8$? I would like to get a sense of whether we can "afford" to use estimates for the prime counting function rather than exact counts. Because of the work of Buthe, we can estimate this function

to square root accuracy (i.e. Riemann hypothesis levels of accuracy) for N as large as 10^{19} [basically this is due to the numerical verification of RH up to a certain large height]. In fact the error in $\pi(N)$ is basically $\sqrt{N}/\log N$, so for $N = 3 \times 10^8$ the error is something like 10^3. If the surplus is comfortably larger than this (say 10^4) then we should have some margin to use approximations for the prime counting function at the very largest scales. This may become increasingly useful as we go to even higher scales than 3×10^8. (There are also some good estimates for the prime counting function beyond 10^{19}, for instance of Dusart, but I think the analytic approach will be able to cover this range and so these results will not be needed for the numerical analysis (and in any event I doubt the numerical methods could ever get to 10^{19} without some serious compute and/or speedup tricks).)

 **Matthieu Rosenfeld** · 6 April 2025

In fact, I was able to use a machine with more RAM and that the $t(n) \geq n/3$ conjecture is verified for n in $[8 \times 10^4, 10^9]$.
For $n = 10^8$, the greedy algorithm achieves a surplus of 252799 (that is, it factors $10^{(8)!}$ in 100252799 factors larger than 10^8). It seems large enough for what you have in mind, I suppose that in order to keep using Ventullo's idea we need to keep some of this margin. Suppose I chose epsilon to be 2/100 (that is we take care of the interval [n, n*1.02]), then we find 100131496 factors of size at least (1.02*n/3), so we still have a decent margin.
It is not completely clear to me how to use this. If I understand, the idea would be to use the bound $0.94\sqrt{n}$ mentioned in to bound the contribution of large enough primes factors to allow us to focus on only the smaller primes ?

 **Terence Tao** · 6 April 2025

Yes, that is the basic idea. From equation (1.8) of Buthe's paper and the triangle inequality one has

$$\pi(x) - \pi(y) \leq \pi^*(x) - \pi^*(y) \leq \int_y^x \frac{dt}{\log t} + \frac{\sqrt{x}}{\log x} + \frac{\sqrt{y}}{\log y}$$

whenever $2 \leq y \leq x \leq 10^{19}$. So, up to terms much smaller than the available surplus, we can approximate the number of primes in an interval $y < p \leq x$ by the logarithmic integral $\int_y^x \frac{dt}{\log t}$ (there is also a matching lower bound, but it is the upper bound which is relevant for our application). So for instance we can use this to upper bound how many primes dividing $N!$ there are between t and N (keeping in mind that primes between $N/3$ and $N/2$ need to be counted twice, etc.), avoiding the need to count them directly. Similarly for the primes between $t/2$ and t, each of which removes one factor of 2 from the factorization of $N!$, and so forth. Because this is a slight overcount of the number of primes, this "virtual greedy algorithm" procedure may remove a few too many factors of 2 etc. from $N!$ when compared against the actual greedy algorithm, but because of the large surplus, we should still be able to locate well over N factors of this size,
This may become important for larger values of N, where one may not have enough memory to store the entire table of primes, though perhaps it is not yet an issue for $N = 10^9$.
I am guessing that the reason why the greedy algorithm slows down when one has very few primes remaining is that there are many fewer available factors, and the algorithm has to increment its way from t upwards for quite a while to find them. One could perhaps precompute a table of smooth numbers (e.g., numbers with all prime factors in

$\{2, 3, 5, 7\}$ that are greater than or equal to t, but with no factor greater than or equal to t), and switch to that table to find factors once these are the only available primes remaining. It may take some trial and error though to determine what is a reasonable definition of "smooth" here.
(Small correction: I believe you meant "larger than 10^8/3" instead of "larger than 10^8" in your previous comment.)

 **Matthieu Rosenfeld** · 7 April 2025

I am not sure how to use an upper bound on $\pi(x) - \pi(y)$. The idea, as I understand it, would be: we simply ignore prime larger than say n/K, and for all $i \leq K$:

- we add $3i(\pi(n/(3i-1)) - \pi(y/(3i)))$ to the number of constructed factors larger than $n/3$,
- and we remove from the remaining available primes factors the same number of occurrences of i (or its factorization) (all of this is up to $+1$ or -1).

So it seems a lower bound on $\pi(x) - \pi(y)$ would be more useful. Am I missing something on how to use the upper-bound ?
Either way, on a powerful machine, storing the primes is not a problem yet (the 4×10^9 primes below 10^11 would still easily fit in "only" $32GB$). Memory-wise, the main problem seems to be the precomputation of the factorization of every integer. Currently, for each n, I only store the smallest prime divisor (or -1 if n is prime, so we always store something smaller than $\sqrt{n}$, hence on 32 bits instead of 64). For 10^11 this would still take $400GB$ which is really near the limit of reasonable machines.
Inspired by this "smooth number" idea, I might have a way to reduce the number of integers whose factorizations need to be kept. Suppose during the greedy procedure, the largest available prime is p, and we try to multiply it with a number z such that $z \times p \geq n/3$. Let $s(z)$, $L(z)$ be respectively the smallest and largest prime divisors of z (I do not know if there are standard notations for these). Then $\frac{z}{s(z)} \times p < n/3$ (otherwise we would have picked $\frac{z}{s(z)}$). On the other hand, since p is the largest available prime, we know $L(z) \leq p$, which implies $\frac{z}{s(z)} \times L(z) < n/3$. So during the greedy procedure, we only need to know the factorizations of numbers that respect this condition (based on some experimentations, for $n = 5 \times 10^7$, this represents roughly 10^6 integers). There might be more clever ways to eliminate "useless" integers that I do not see.
Based on one of your comments of March 29th, I was under the impression that we want to reach a value near 10^11 where the asymptotic argument from your preprint takes care of the rest. It really seems doable with the current approach on a big machine with a few extra optimizations. Also since each factorization covers an interval of the form $[n, 1.02n]$ (maybe for different values of 1.02), then the certificate of the verification for the interval $[10^6, 10^11]$ is the factorization of only 600 numbers (although, I am not sure that it is easier to verify the certificate than to produce it).
(Yes, I meant 10^8/3.)

 **Terence Tao** · 7 April 2025

Ah, you are right, both upper and lower bounds are needed: lower bounds to ensure that we have produced enough terms in the factorization, and upper bounds to ensure that we have not "spent" more small primes than are actually available in the factorization. Fortunately, the results of Buthe also provide lower bounds of essentially the same shape as the upper bounds (there are some minor changes in the constants and some lower order terms). Equations (1.9) and (2.0) gives the lower bound

$$\pi(x) - \pi(y) \geq \int_y^x \frac{dt}{\log t} - \frac{\sqrt{x}}{\log x}(1.95 + \frac{3.9}{\log x} + \frac{19.5}{\log^2 x})$$

which is slightly less elegant than the upper bound, but no problem for a computer of course (for fast computation one would probably want to use some asymptotic approximation of the logarithmic integral with known bounds on error terms, but these can be found in the literature, and may even already exist in code libraries). But it sounds like they won't be needed just yet, unless we want to push up by another order of magnitude or two.

I like your idea of pruning the table of available integers to only keep the "non-useless" ones. I had a further idea that looks easy to implement, and performs a tradeoff between optimality of the factorization, and the speed of the factorization. Currently, when the largest available prime is p, we search for complementary factors $z \geq t/p$ that are still available (after removing p from the available pool of primes remaining), usually taking the smallest such value, to join with p to create a new factor in our factorization. When p is small, then t/p is large, and so we have a large search space to cover, which is presumably taking up most of the run time. But suppose we select an intermediate parameter $1 \leq K \leq \sqrt{t}$ and modify the greedy algorithm as follows: greedily select the largest primes $p_1, p_2, \ldots$ available until we reach a product $p_1 \ldots p_k$ that is at least K (and will therefore be at most K^2, unless it consists of a single large prime p), then match it with a complementary factor greater than equal to $t/p_1 \ldots p_k$. This threshold is at most t/K, so by taking K larger we have to store fewer factorizations, and perform faster searches. When the largest prime p is at least K, this modified algorithm is identical to the greedy algorithm; but when one is down to the primes smaller than K it is a little more inefficient, though the threshold $t/p_1 \ldots p_k$ we are searching at is still relatively large (it doesn't go below t/K^2) so as long as K is not too close to $\sqrt{t}$, it should still give a reasonably efficient factorization. So one has an additional parameter K to play with: $K = 1$ is the vanilla greedy algorithm (gives a good surplus but is slow), and increasing K trades off the surplus for speed.

I'm glad it looks like getting up to 10^{11} is in fact feasible. I'd been postponing working out the details exactly, because working with completely effective constants everywhere is something of a pain, but now that we seem to be approaching that scale, I should go work out the details soon.

A reasonably standard notation for $s(z), L(z)$, by the way, are $P_-(z)$ and $P_+(z)$ respectively.



uhrmar · 6 April 2025

I think you have to be careful with how many problem instances you skip when solving problems with higher threshold. In particular, it also depends on the number of factors you find. If you solve for N and $t(N) = N(\frac{1}{3} + \epsilon)$ and you find $N(1 + \delta)$ factors (say), this verifies only the interval $[N, N \cdot \min(1 + 3\epsilon, 1 + \delta)]$. Ideally, $\epsilon \approx \delta$.

Plugging in the numbers $N = 300000$, $\epsilon = 0.01$ ($t(N) = 103000$) finds 299733 factors, making δ negative and the interval empty.

(I hope I got this right.)



Terence Tao · 6 April 2025

As you go from $N!$ to $(N+1)!$, you can add one factor for free, namely $N+1$. So as soon as you can (sub)factor $N!$ with at least N factors that are greater than equal to $N(1/3 + \epsilon)$, then you can also factor $N^{(\prime)!}$ into N' factors that are greater than or

equal to N' for any $N \leq N' \leq N(1+3\epsilon)$. So I don't think it's necessary to take δ into account (other than to ensure that it is nonnegative).



uhrmar · 6 April 2025

Please disregard what I wrote in the previous comment – I got it completely wrong.



Terence Tao · 4 April 2025

Looking at the code, it seems the strategy is to view the factorization as $\prod_{t \leq j \leq N} j^{x_j} = N!$ (actually it is slightly better to work with subfactorizations $\prod_{t \leq j \leq N} j^{x_j} \mid N!$) for some natural number variables x_j, in which case the problem of factoring $N!$ to as many factors $\geq t$ as possible amounts to maximizing $\sum_{t \leq j \leq N} x_j$ subject to the constraints $\sum_{t \leq j \leq N} \nu_p(j) x_j \leq \nu_p(N!)$ for every prime $p \leq N$. It is somewhat impressive to me that the linear programming package you use (Gurobi) can handle programs of this size (for $N = 300000$, this is 35997 equations in 200000 unknowns). Still, it seems to give very good results, and even tight upper bounds. It should be very helpful in determining $t(N)$ exactly for small N (e.g., the range $80 \leq N \leq 599$ depicted in my paper).

There are some small ways to reduce variables which may improve the runtime of the linear program (at the cost of making the code a bit more complicated). Any even number j with $j \geq 2t$ can be deleted, since there is no harm in replacing j by $j/2$; this already gets rid of 50001 unknowns in the $N = 300000$ case. Any prime p with $N/2 < p \leq N$ appears exactly once in $N!$, and the only j that is usable here is p, so this forces $j_p = 1$. Similarly we must have $j_p = 2$ for $t < p \leq N/2$ (assuming $t \geq N/3$). Once we make these reductions, we no longer need to test primes larger than t; for $N = 300000$, this reduces the number of equations from 35997 to 9592. (But it is possible that Gurobi is already smart enough to find these sorts of initial reductions on its own, and the actual speedup in practice may be slight.)

EDIT: there is one subtlety in using the linear programming method to rigorously demonstrate an upper bound on $t(N)$. As currently implemented, the method assumes that all factors of $N!$ involved are not only greater than or equal to t, but also less than or equal to N. This is a pretty reasonable assumption: if one had a putative factorization (or subfactorization) that involved a factor larger than N, in most cases one can perform some sort of "trade" in which one decreases that number and increases some other one in a way that moves the subfactorization more towards being having all terms bounded by N, but I don't see yet why this is possible to do in all cases.



Terence Tao · 4 April 2025

OK, here is a possible way to provide a fully rigorous demonstration of an upper bound for the number of factors greater than or equal to t in a subfactorization of $N!$. There is a dual linear program to the primary one: if one can find non-negative reals $a_p \geq 0$ for primes $p \leq N$ such that $\sum_p a_p \nu_p(j) \geq 1$ for all $j \geq t$, then one has the upper bound $\sum_{j \geq t} x_j \leq \sum_p a_p \nu_p(N!)$ to the primary problem. So one just has to find a solution to the dual problem with $\sum_p a_p \nu_p(N!)$ as small as possible.

The catch is that there are infinitely many j's. But if we impose the additional (presumably reasonable) constraint that the a_p are weakly increasing, then for $j > N$, we can use Bertrand's postulate to reduce one of its prime factors of j by a factor of at most two to obtain $t \leq j' < j$ with $\sum_p a_p \nu_p(j) \geq \sum_p a_p \nu_p(j')$. So if we solve the finite linear program $\sum_p a_p \nu_p(j) \geq 1$ for all $t \leq j \leq N$, with the additional constraint that the a_j are weakly increasing, this should give a rigorous upper bound that I expect to be close (or exactly equal to) what the previous not-quite-rigorous linear programming bound gave.

 **uhrmar** · 5 April 2025

I also think ignoring factors $j > N$ is the only way the LP upper bound could fail. The argument with the weakly increasing dual variables is very elegant. Just to make sure I correctly understand it: the argument only works for $t(N) \leq N/2$ since one has to apply it repeatedly to end up with $t \leq j' \leq N$ (the base case verified by linear programming). As such it might be worth pointing out that only replacing primes with smaller nearby primes might get stuck at powers of two. But it is also possible to just drop factors of two.
I've opted to implement this in my primal formulation by adding additional variables that force the optimal dual variables to be weakly increasing. Unfortunately, adding these variables might destroy the validity of the factorization – it is only useful to verify the upper bound. I've also added a test of monotonicity of the optimal dual variables to the original formulation. It seems that the optimal duals are very often (always?) weakly increasing.
It *feels* like one should be able to prove the weakly increasing property of the simple formulation, but I'm unable to do so.
If I find time, I might implement a verification of solutions found by Gurobi using rational arithmetic. This would make the code more trustworthy for extending OEIS A034258.

 **Terence Tao** · 5 April 2025

Yes, the argument assumes $t(N) \leq N/2$, but this can be proven from the trivial bound $t(N) \leq (N!)^{1/N}$ for $N \geq 6$ (and is also true for $N = 2, 3, 4, 5$. You're right that I neglected to treat the case of powers of two, but as you say that case can also be handled.
Regarding rational arithmetic, if there is some room in the linear programming bound (e.g., if we want to prove $t(N) = 45$ and the linear programming upper bound is $45.943\ldots$) then one could simply round up the weights a_j to the nearest multiple of $1/1000$ or something and verify using integer arithmetic. This might be good enough for the small values of N needed to extend A034258, and one simply needs to report the weights to this level of precision.
I can't yet prove the weakly increasing property for the finite linear program (with $t \leq j \leq N$), but I can do it for the infinite linear program (with $j \geq t$). Suppose for contradiction that the optimal weights a_p for the infinite linear program obey $a_{p_1} > a_{p_2}$ for some primes $p_1 < p_2$. Let a'_p be the same weights as a_p, except that a'_{p_1} is assigned to equal a_{p_2} rather than a_{p_1}. I claim that these new weights still solve the linear program $\sum_p a'_p \nu_p(j) \geq 1$ for all $j \geq t$. This is because $\sum_p a'_p \nu_p(j) = \sum_p a_p \nu_p(j')$ where j' is formed from j by replacing any factors of p_1 with p_2. So these new weights are admissible, but on the other hand $\sum_p a'_p < \sum_p a_p$, contradicting the optimality hypothesis.

 **dragonbriskly18bd0d56d5** · 11 April 2025

It seems to me that one can prove, in the original formulation, that no factor need be greater than n^2 (or, in fact, nt). Any such factor has a proper divisor which is smaller, but still at least t.
Accordingly, in the primal formulation, it suffices to extend the range of j somewhat higher. Or even better: only consider potentially "useful" j that have the appropriate divisors. Nonetheless, this would make the problem bigger and thus potentially take longer.

But: Do I reason correctly, then, that in the dual formulation, it suffices to check j up to this same bound (say n^2 for simplicity), even if the a_j are *not* sorted?



Terence Tao · 11 April 2025

Nice observation! Yes, this would allow for a rigorous dual certificate without requiring the a_j to be non-decreasing, as long as it covered all possible j.

There is a way to cut down the set of j significantly, but it is complicated. The first observation is that if a factorization involves two factors j_1, j_2, and j_2 dominates j_1 in the sense that there exist factorizations $j_1 = k_1 l_1$ and $j_2 = k_2 l_2$ for which $k_1 < k_2$ and $l_1 < l_2$, then it is possible to interchange these two factors to $k_2 l_1$ and $k_1 l_2$ instead, which reduces the variance of the logarithms of the factors. So without loss of generality, if one restricts to factorizations (or subfactorizations) that minimize variance (for a fixed value of total product), one can assume that no such domination occurs.

To use this, we make a second observation: if there is a (sub)factorization $\prod_{j \geq t} j^{m_j} \mid N!$ with at least N elements then we must have $\sum_{j \geq t} m_j \log \frac{j}{t} \leq \log N! - N \log t$. This puts a cap on having too many factors j in which $\log \frac{j}{t}$ is large.

One can combine these as follows. Suppose that we have a factorization that contains a large factor $j = kl$ with k, l both large. Let p be a prime with $e^\sigma t/l \leq p < k$ for some $\sigma > 0$. Then, in order to avoid domination, the only factors that contain p, must be the product of p and a quantity which is at least l, and so will contribute at least σ to the sum $\sum_{j \geq t} m_j \log \frac{j}{t}$. On the other hand, as all factors have size at most nt, the total number of factors of this form is at least $\frac{1}{\log(nt)} \sum_{e^\sigma t/l \leq p < k} \nu_p(N!) \log p$. So we get a contradiction provided that

$\sup_\sigma \frac{1}{\log(nt)} \sum_{e^\sigma t/l \leq p < k} \nu_p(N!) \log p \times \sigma > \log N! - Nt$.

This criterion in principle rules out a lot of large factors $j = k \times l$ from consideration; but the amount of effort needed to test this criterion may be comparable to the time needed to just run the larger linear program.



Matthieu Rosenfeld · 4 April 2025

I have written a version of my code [https://gite.lirmm.fr/mrosenfeld/guy-selfridgeconj] that allows to run the heuristic for all values between a given range with a few different parameters for each value. In the range [100000,500000] the largest integer that fails is 138074 and there are only 18 of them failing in the range. On my laptop, it currently runs in under ~70 minutes.

It seems that the linear program based solution is a bit too slow to run it on all integer up to 500000 (it takes ~10m on my machine, although a big fraction of the time seem to be spent on initializing the LP ?). It probably makes sens to improve the heuristic to reduce the question to as few cases as possible before running the LP based solution for the remaining ones.

PS: This is not the deepest remark, but it took me a while to notice that it is enough to verify the conjecture for all multiple of 3. In particular, there are in fact only 6 missing values in [100000, 138074]:100014, 100017, 100020, 100023, 101790, 138072



Terence Tao · 4 April 2025

Thanks for the additional runs. It seems your method, though less powerful, is significantly faster than the linear programming method, so perhaps it makes sense to do some sort of two-pass search where one uses the faster method to eliminate most candidates and then the slower one to handle the few remaining exceptions.

One can also consider a three-pass search in which one first uses the faster method with an aggressive parameter selection heuristic to achieve an initial winnowing, and then again

with a more conservative heuristic on the remaining candidates, and then finally the linear programming method on the remainder, but perhaps this is overly intricate.
It makes sense to restrict attention to multiples of three throughout.

 **uhrmar** · 4 April 2025

Yes, setting up the linear program was extremely inefficient. I've pushed a fix that makes the code run significantly faster.

 **Anonymous** · 4 April 2025

In the paper, in the definition of t(N), it is not clearly stated that the factors should be integers
[Fair enough, I will add that $a_1, \dots, a_N \in \mathbf{N}$ in the next revision of the ms. -T]

 **Anonymous** · 4 April 2025

If the factorization of N! is to arbitrary poditive numbrs than t(N) should be the geometric mean of 1, ...,N (which is assymptotic to N/e)

 **Terence Tao** · 8 April 2025

I started a Github repository to record the various progress towards the Guy-Selfridge conjecture, with an aim to coordinate future work. I am also beginning to draft some notes on how to numerically generate upper and lower bounds on t(N) within the repository, though this is currently a work in progress (in particular I have some ideas on how to improve and make explicit the asymptotic construction, and hopefully meet the numerical computations somewhere in the middle).
As listed on the repository, there are various directions to pursue here. The major one is to complete the verification of the Guy-Selfridge conjecture $t(N) \geq N/3$, which is currently known for $8 \times 10^4 \leq N \leq 10^9$ and for sufficiently large N. Actually, the lower bound may be close to optimal, given the predicted asymptotic $t(N) \approx N/e - c_0 N/\log N$ with $c_0 \approx 0.3044\dots$, as this approximation meets $N/3$ at about $N = 6700$. (I've also listed the more accurate evaluation of c_0 as a further numerical goal.) A further goal is to extend the OEIS table for $t(N)$, which currently goes up to $N \leq 79$.

 **Boris** · 8 April 2025

Thanks for starting the GitHub repository. A couple comments, which I didn't know whether to raise here or as an issue there:
1. The file https://terrytao.wordpress.com/wp-content/uploads/2025/03/tbounds.txt which is linked from the bottom of the README.md as "Lower bounds on $t(N)$ for $80 leq N leq 599$" is inaccessible to me. The message is "— 403: Access Denied — This file requires authorization.: You must be logged in and a member of this blog."
2. You currently report the OEIS entries as known up to t(79)=22, which is indeed as far as https://oeis.org/A034258/b034258.txt goes. However, https://oeis.org/A034259/b034259.txt is a different representation of the same data and there it goes up to t(200)=59.
[Thanks, these issues have now been corrected – T.]

Decomposing factorial of 300K as the product of 300K factors larger than 100K - Gustavo · 8 April 2025

[...] few days ago, Terence Tao proposed a challenge to decompose the 300K! as the product of 300K factors larger than 1.... A smaller example is decomposing 10! as the product of 10 factor greater or equal than [...]

Daniel Eduardo Ruiz · 8 April 2025

Trivially $\min a_j \leq \Omega(N!) = n\ln(\ln(n)) + Cn + O(\frac{n}{\ln(n)}$. Omega's function counts repetitions. It looks like VLSM.
Naive algorithm:
0. n_i<-N!, T_n=0
1. Take min a_i = Omega(n_i) and divisor of N_i.
2. n_i /= a_i
3. T_n ++
4. Repeat until a_i > 1 or something like that.
It would be interesting to combine with the result:
$\left(1-\frac{\pi^2}{12}\right)\frac{N^2}{\log N} + C\frac{N^2}{\log^2 N} + o\left(\frac{N^2}{\log^2 N}\right), \quad C \doteq 0.1199$
The source is Properties of sum-of-digits functions page 55. Or full article Digit sums and the number of prime factors of the factorial n!=1·2⋯n, and also have

Terence Tao · 10 April 2025

I've found a way to make the constructions in my paper more efficient, and in particular a viable path to providing fast factorizations of $N!$ for say $10^9 \leq N \leq 10^{11}$, using 3-smooth numbers (products of 2 and 3) rather than just 2-smooth numbers (powers of 2) for improved efficiency.
The algorithm is described in Section 5 of these notes, but roughly speaking it proceeds as follows. One first starts with an approximate factorization formed by taking all the numbers between t and $t(1+3t/A)$ that are coprime to 6, with multiplicity A, where A is a parameter that is not too large. This gives a factorization that uses up about the right number of medium primes, but has the "wrong" large primes, and uses none of the available primes at 2 and 3. Then one deletes all the factors with large primes p (say larger than t/K for some parameter K a bit larger than A) and replaces them with what the greedy algorithm says to use, namely $p\lceil t/p\rceil$. Then, for all remaining primes between 3 and t/K, one fixes up the factorization by rounding any excess primes up to the nearest 3-smooth number (products of 2 and 3), and also filling in any missing primes by a greedy argument also using 3-smooth numbers. After all this, one will be left with a large number of powers of 2 and 3, which one can efficiently factor using a simple 2-dimensional linear program.
I believe that this construction is slightly less efficient than the greedy algorithm, but is relatively straightforward to analyze theoretically, and I think also should be fairly quick to implement numerically as most of the primes are already allocated automatically, and only a small amount of "cleaning" is needed.
As an application of this new construction, I believe I can improve the lower bound $t(N)/N \geq 1/e - O(1/\log N)$ to $t(N)/N \geq 1/e - c_0/\log N - O(1/\log^{1+c} N)$, thus matching the upper bound. I plan to write up the details of this soon. After that, I will try to make everything explicit and see exactly what range of N I can cover with the theoretical approach.

> **Matthieu Rosenfeld** · 11 April 2025
>
> I have been rewriting my code to use the optimization I mentioned in the previous message (store only the factorization of the "useful" integers). I also had to rewrite the sieve for this, and to turn it into a segmented sieve (takes $O(n/\log n)$ space instead of n). Lot of other small implementation details, means I spent more time than expected to implement this new version. But, yesterday I was able to run it for up to 2×10^{10} in 2h45m using $70GB$ of RAM (memory consumption is sublinear, and the machine has 300GB of RAM, so I should be able to push it at least to 8×10^{10}). I will try to make a pull request, for the C++ code and the output after cleaning-up and running the code for 8×10^{10}.

If needed, using the trick of working with approximation of $\pi(i)$ for $i \leq t/3 = N/9$ should allow us to reach 10^{12}. Memory is the main limitation, so we could precompute and store the exact value of $\pi(K*i)$ for all $i < N/K$ for some K 10000, this would only take $O(N/K)$ bytes but provide good approximations on π. (I could also use even more powerful machines with up to 3TB of RAM, but it should be only for final computation as there are more complicated to access. It would also make the computation more difficult to check).
For the approach you suggest in section 5 of the notes, there are some implementation details, that I am not sure how to implement if we want to reach values $\geq 10^{11}$. For part (6), we need to know the factorization of all numbers in $[t, t(1 + 3/A)]$. I believe that it is too expensive to compute it as you need it for each number (I might be wrong ?), and on the other hand precomputing it with a sieve will be too expensive memory-wise. It is really nice to see that it improves the theoretical bounds, though.



Terence Tao · 11 April 2025

That's great news! I am still in the process of working out exactly what the construction I have will give (and the notes will be fluctuating a fair bit doing this process, for instance the algorithm has currently been moved to Section 3.2), but I have several ideas for upper bounding various quantities coming from things such as the prime factorizations in $[t, t(1 + 3/A)]$ that should allow for a more computationally feasible test of the algorithm I propose. But it is still a work in progress, I will update here when there is a concrete criterion that can be tested numerically.



Kevin Ventullo · 12 April 2025

Matthieu,
Earlier you mentioned that storing the smallest prime factor for every $n < 10^{11}$ would take up about 400GB of memory. Naively, it seems to me you could halve that by only storing the values for odd n; computing divisibility on the fly for p=2 is extremely cheap.
More generally, there is an approach that would let you index only those n for which the smallest prime factor is greater than say, 7. This already brings the array's size down to under 100GB. It would mean doing O(1) runtime multiplications and managing a few small auxiliary arrays instead of a direct lookup in firstDivisor and nbPrimeDiv, but maybe this is worth the overhead.
Happy to expand on this and write up some code if you're interested.



Evan Conway · 13 April 2025

I think this is a good approach, given that many small divisibility checks are extremely cheap.
In terms of implementation, would you first do divisibility checks against the first k primes and then proceed to doing a lookup in firstDivisor? If so, then you could potentially do an optimization where you store a single bit for all $n < 10^{11}$ numbers saying if it has a small smallest factor (do the manual divisibility checks) or a large smallest factor (lookup in firstDivisor). This would add an array of size 12.5 GB, but I think it would ultimately decrease memory usage, since you can add more manual divisibility checks without decreasing performance too much.
Working through an example of this, suppose we do manual checks for the first 10 primes. All but around 15.8% of numbers have one of these primes as their smallest factor, so we would need around 400 * 15.8% = 63.2 GB to store the direct lookup array. We then add 12.5 GB for the helper array I'm proposing, for a total of 75.7 GB.

Then, for each lookup, we run a lookup on the helper array. 15.8% of the time, we then just run another lookup on the direct lookup array. The rest of the time, we try divisibility checks for the first 10 primes. I think this should actually be fairly cheap (I'm getting around 2 expected checks, if my calculations are right), since we're likely to find the smallest prime factor within the first few checks.



Matthieu Rosenfeld · 13 April 2025

This is a really reasonable trick that should allow to go a bit further. I haven't had time to clean up my code. But it was easy to make a small modification that does this trick for even numbers. I started a computation with 10^{11}. Adding other primes has diminishing returns, but also taking care of even numbers is enough to allow us to reach 10^{11}. Also, the numbers whose factorization is useless by the previous criterion I gave tend to have small factors, so it might be better to try this trick without the trick hat I added to my code ? Maybe in that case you want to precompute an array that tell you what to do based on N modulo K (where K is the product of the 10 first primes).
I hope that I will be able to share a reasonable version of the code tomorrow. I suspect there is something a bit inefficient with the sieve I have written, as the precomputation is the most expensive part now.



Kevin Ventullo · 13 April 2025

For p=2, you check directly. For other primes, there are two more issues to consider:

1. In addition to the smallest prime divisor of a given number, Matthieu is also interested in the number of times that prime divides the number. This information is stored in the array nbPrimeDiv. For p=2 this can also be cheaply computed on the fly by counting trailing zeroes.
2. The indexing scheme for firstDivisor and nbPrimeDiv gets a little weird when you only want to index numbers which are coprime to the first k primes.

To resolve this, let P_k denote the product of the first k primes, ignoring 2. My thinking was you could do a single multiplication to compute the remainder of n modulo P_k, and then use a precomputed auxiliary array of length P_k to store either the smallest divisor when n is not coprime to P_k, or indexing information when n is coprime to P_k (as well as metadata about which of the two situations we're in). When n is coprime to P_k, one more multiplication is required to compute the exact index into firstDivisor and nbPrimeDiv.
In any case, that gets you the firstDivisor information in just two multiplications, independent of k.
There's a bit more work to compute nbPrimeDiv when n is not coprime to P_k, which requires at most three more multiplications (so at most four in total) and some small-ish auxiliary data. Basically, the idea is to bring back the arrays firstDivisor and nbPrimeDiv, but only for multiples of p^5 for each p in the first k primes (ignoring p=2). The size of this array shrinks rapidly: for p=3, the array is ~1.6GB, and for p=5 it is ~128MB. The multiplications come from checking divisibility by p^5, and then a tiny binary search when the exponent is less than 5. Note that when the exponent is >= 5, only two multiplications are required in total.
Now, this is not really viable if you take k=10, as then P_k is rather large. However, I think you hit diminishing returns well before 10. As you observed, k=10 gets you to ~16%, but even k=4 already gets you to ~23%. In any case, there some knobs to fiddle which trade off memory usage with runtime compute cost.
Footnotes:

> As for how to turn exact division and remainder calculations with compile-time constant divisors into runtime multiplications, see https://lemire.me/blog/2019/02/08/faster-remainders-when-the-divisor-is-a-constant-beating-compilers-and-libdivide/

 **Matthieu Rosenfeld** · 14 April 2025

I have added the last source code to my git repo [https://gite.lirmm.fr/mrosenfeld/guy-selfridgeconj/-/blob/main/RangeOfValuesWithEpsilonSpecialDivisors.cpp?ref_type=heads]. I have tried to add some comments to make it more readable, but I might still be missing some comments. I will try to learn how to do a pull request this afternoon (europe afternoon).
I have also added to this git the output of the run with 10^{11} (with the output of /bin/time -v in the file aswell have time and memory consumption). It ran for 15h and used 218GB of RAM. Theoreticaly the running time should be sublinear: for a given integer n it should be proportional to πn, and we need to treat fewer number in $[10^{10}, 10^{11}]$ than in $[10^9, 10^{10}]$ because the epsilon of margin is larger. It is also sublinear in practice (which is a surprise to me, as I was expecting cache misses and other problems to be really compensate): it takes ~6 minutes for 10^8 which is roughly 150 times less than for 10^{11}. Anyway, will all the tricks mentionned in previous comments, and the fact that we can still use even more powefull computers, we can still go a bit further than 10^{11} with this implementation if needed.
Note that if one starts with 10^7 the heuristic becomes almost useless. It cost the same amount of time to try the heuristic with different parameters than to simply decrease the value of epsilon until it works. On the other hand the array that keeps the number of primes in the factorization of a number is only usefull for this heuristic. So I simply removed the heuristic from this version of the code (it also simplifies the code a bit).
Interestingly, the greedy procedure finds a solution with epsilon = 60 at almost every value after 10^10. This means we find n factors of size at least $(n \times (1000 + 60)/1000)/3$ (keep in mind that all the divisions are integer divisions). $3/1.06 = 2.83$ which is not much larger than e.
I do not know when it starts to find solution with epsilon = 70, as it it currently not one the considered parameters.
other implementation details for those who might look at the code (this also adress some details from the few previous comments):
* I am using cc_hash_map they are more efficient than stl::unordered_map, but they might not be available if you are not using g++ on a linux machine (as far as I understand the ext/pb_ds/assoc_container.hpp is part of gcc/g++, but I am not sure...). Also they seem to occupy more memory than stl::unordered_map, so if needed we could simply replace them to go a bit further. In fact, someone more competent with hashmaps, might be able to replace it with a well chosen hashmap for the current usage...
* A large fraction of the computation time is spent on the precomputation. I suspect that this might be because writing in a really large hashmap is expensive (lots of cache misses ?). There might be easy trick that I do not know to make this part more efficient (without implementing more advanced sieves than Eratosthenes').
* behavior for 10^9 and 10^{11} is really different, so verifying if a small change improves something require to run the computation on large values and takes a lot of time.
* Although I have a couple of sanity checks, I cannot yet claim with sufficient certitude that there is not small glitchs in this code.

>  **Terence Tao** · 14 April 2025
>
> That's great news! I've added some comments on your associated pull request for some possible next steps (some of which would help with the "sanity checks").

There has been a fair bit of progress on the github, by the way, beyond what is reported in the blog comments here; for instance, there is now a separate proof of Conjecture 2 in the asymptotic regime, although it is currently not clear how effective it can be made. I am getting close to completing the effective analysis of my own asymptotic construction, although it was messier than I expected; in the end, though, it should not involve any actual counting of large primes, as the effective prime number theorems in the literature appear to be sufficient to control error terms in ranges such as $N \approx 10^{11}$, although I am yet to fully identify a set of parameters in which all the numerical conditions required for the construction to actually certify that $t(N) \geq N/3$ are actually valid in this range. Will report later when I have some more conclusions.

 **Terence Tao** · 15 April 2025

Very provisionally, I found some parameter choices in my construction that resolve Conjecture 2 for $N \geq 10^{13}$, two orders of magnitude higher than I expected. Am trying to tinker with the argument to see if one can do a little better than this.

 **Terence Tao** · 17 April 2025

After making explicit estimates of all error terms (which was significantly more tedious than anticipated, especially when trying to find ways to make the estimates not lose factors of two etc.), I was able to arrive at a numerical criterion for verifying Conjecture 2 for $NgeqN_0$ for any given N_0, provided that one supplies some supplemental parameters A, K, L that verify some complicated numerical inequalities. With a little bit of python and some trial and error I could verify these conditions for $N_0 = 10^{12}$ with very little room to spare; so there may be a little bit of leeway to push this down, but probably not all the way to meet the greedy algorithm at 10^{11} unfortunately. The theoretical justification of the criterion is in the draft version of the paper here, though I need to proofread it further.

I will have to turn attention to other matters for a bit, but hopefully only a small amount of additional effort will be needed to fully settle the Guy-Selfridge conjecture.

 **Evan Conway** · 18 April 2025

Since we now need to extend this to reach 10^{12}, here are some questions / comments I had on improving the code:

- Is there a particular reason the code using a hashmap rather than an array? I was looking at accesses and it looked like every number in the range was being accessed by the sieve, so I'm unclear why an array wouldn't be used instead.
- For the sieve, I agree that there's a good chance that that's being slowed down by cache issues. Currently, it looks like the code is looping through primes one at a time (2, 3, 5, etc) and then going through and marking down all those multiples. I would imagine this would lead to a large number of cache misses. My thought is to switch to an array (as mentioned in the last bullet) and then sieve each block one at a time, sieving all numbers in that block. This way you only access each block once.
- This seems to have some good details on fast prime sieving: https://github.com/kimwalisch/primesieve/blob/master/doc/ALGORITHMS.md
- I notice that currently, 2 is one of the primes that is sieved. I'm unsure how much time this takes up, but one possible optimization is to use the optimization Kevin Ventullo suggested and just handle checking for divisors of power 2 directly, when you're actually running the greedy algorithm.
- I'm wondering if you can exploit the jumps of $N * (epsilon / 1000)$ to decrease the number of factorizations you need to store. Specifically, I was thinking that instead of jumping by

$N * (epsilon / 1000)$, you could jump to the first multiple of 1024 (for example) before $N * (1 + epsilon / 1000)$. The idea here is that you could then just check $N / 1024$ to get the remaining factorization. If I'm thinking about this correctly, I believe you could get a very large decrease in the number of factorizations you need to store, at a relatively low cost.

- You've probably already accounted for this, but what compiler flags are you using? If you're not using -O3 and -march=native, that often gives a decent constant-factor speedup.

Please let me know what you think of this. Also, take this all with a grain of salt, since I haven't had a chance to properly look at what's taking up the most time in the code (premature optimization is the root of all evil, etc).



Anonymous · 18 April 2025

It might be worth switching languages away from python at this point. Julia might be a good candidate here as it will have much better runtime than Python, good datastructes and the Primes.jl package which has very fast primality testing and factorization of small numbers, and a pretty decent prime sieve. I'd be happy to help with the translation if desired.



Boris · 15 April 2025

Here's a wacky idea. I don't know whether it goes anywhere. We begin with the (integer) linear programming formulation as described in this comment: https://terrytao.wordpress.com/2025/03/26/decomposing-a-factorial-into-large-factors/

It is to find natural $\{x_j\}_{j=t}^N$ maximizing $\sum_j x_j$ subject to the constraints $\sum_j v_p(j)x_j \leq v_p(N!)$ for each prime p. This is a faithful translation of the original problem (assuming the factors don't exceed N).

What if we make the problem harder? What if we instead have constraints for each prime power q that $\sum_{q|j} x_j \leq N/q$? By summing over all powers q of the same prime p (and taking a floor on the RHS), we would recover the original constraints, so this is a harder problem.

I ran some experiments and I find that solutions with $t = N/3$ seem to exist for $N \geq 50000$ or so, perhaps only slightly worse than the original Conjecture 2. ($N = 43632$ does fail.) I would boldly conjecture that for this problem, it's also the case that $t(n)/n \to 1/e$...



Anonymous · 18 April 2025

Some brief comments regarding computing the constant c_0. Split the floor function $\lfloor y \rfloor = y - \{y\}$ to get two different integrals.

The integral with y can then be written, essentially, in terms of the infinite sum $\sum_{k=1}^{\infty} \log^2(1 + 1/k)$. This infinite series can basically be computed to any desired precision by series acceleration techniques, for instance Kummer's transformation.

The integral with the fractional part $\{y\}$ is still tricky, but here one has gained a factor of y^{-1} when trivially bounding the tail, so this should allow one to squeeze out some more digits of accuracy.



Terence Tao · 19 April 2025

In the latest version of the draft paper, I have managed to optimize the explicit form of the asymptotic construction to the point where it (barely) works down to $N \geq 10^{11}$, improving from the previous $N \geq 10^{12}$. Subject to checking both the numerics and the theoretical analysis, this now fully resolves the Guy-Selfridge conjectures.

However, the estimates are delicate, and it would be good to have more of a safety margin. I've set the construction up so that one has to verify two numerical conditions $\sum_{i=1}^8 \delta_i \leq \delta$ and $\sum_{i=1}^7 \alpha_i \leq 1$ in order to verify a claim such as $t(N) \geq N/3$. For $N = 10^{11}$, I was able to numerically locate parameters for which $\sum_{i=1}^8 \delta_i \leq 0.9729\delta$ and $\sum_{i=1}^7 \alpha_i \leq 0.9627$; so one wins, but only barely. There

may be some play in optimizing the parameters further to widen the safety margin and perhaps lower the threshold for N somewhat.
One consequence of having such little margin of error is that one has to retain more terms in the effective estimates than I would like; I am accustomed to, say, bounding $X + Y$ by $2\max(X, Y)$ because the factor of 2 is usually something I can afford to lose, but these sorts of simplifying moves are basically not available except for some extremely small lower order terms, and the argument has become quite complicated. Having more of a margin of error would also allow some simplifications to the theoretical analysis.

> **Terence Tao** · 19 April 2025
>
> An unfortunate update: it turns out that the estimates I was using were non-monotone in N; so while they do verify the conjecture precisely at $N = 10^{11}$, they do not necessarily cover the entire range $N \geq 10^{11}$, and similarly for the previous verification at $N = 10^{12}$. (The asymptotic verification for sufficiently large N is still fine, though.) In the efforts to optimize the estimates, I started saving negative terms in order to use them to cancel some positive terms; but this caused the issue that the difference of two monotone decreasing functions is not necessarily monotone decreasing.
> The failure of monotonicity is somewhat mild, but cannot be easily addressed in the $N \approx 10^{11}$ regime where there is so little margin of error available. I will probably have to resort to interval arithmetic to rigorously quantify the errors here.
>
> > **Terence Tao** · 20 April 2025
> >
> > Second update: the interval arithmetic approach works; coverage of the entire region $N \geq 10^{11}$, and the Guy-Selfridge conjectures are once again fully proved (assuming no further bugs in either the code or the theoretical analysis, of course).

Anonymous · 19 April 2025

Is a generalization to z-smooth numbers possible?

1. **z-smooth ceiling (min z-smooth >= x):**

Introduce a cutoff L≤x, e.g. $ latex L = \log ^2 n$, and define:

2. **Rounding-error constant:**
3. **Oscillatory-loss function:**

Lower bound
Construct an admissible factorization using z-smooth numbers:

1. Fix $\alpha < 1$, and set $t = \alpha N$.
2. Choose a smoothness bound z, define $\mathcal{P}_z = \{p \leq z\}$ and $M = \prod_{p \leq z} p$.
3. Pick parameters A and σ such that
   $At \ln(1 + \sigma) = N$ and $\sigma = \frac{\pi(z)N}{At}$.
4. Define the multiset $\mathcal{E}^{(0)}$ to be A copies of all z-rough integers in $(t, t(1 + \sigma)]$.
5. For each prime $p > z$, repair the deficit Δ_p in the factorization by swapping in factors $\lceil p/t \rceil^{(L)}_{mathcal P_z} \cdot t$,
   each costing at most $\kappa_L(z)$ in log-space.
6. For primes $p \leq z$, use the supply from N! to fill in missing p-powers with z-smooth replacements, also at cost $\kappa_L(z)$ per use.
7. The initial log-product of the approximate factorization satisfies
   $\sum_{a \in \mathcal{E}^{(0)}} \ln a \approx N$,
   and by Stirling's approximation, this leaves a small excess log-budget of $O(N/\ln N)$.

8. The total cost to fix all prime discrepancies is $O(\kappa_L(z)N)$, which is $o(N)$ if $\kappa_L(z) \to 0$ (e.g., with $L = \log^2 N$).

Therefore, a full admissible factorization exists, so
$t(N) \geq \alpha N$.
This generalizes the 3-smooth case.



Terence Tao · 19 April 2025

Yes, the method I gave for the 3-smooth case should also work for other thresholds, though I believe the quantitative bounds become worse for medium values of N due to the increased size of the primorial. Asymptotically, this might help improve the convergence bounds on $t(N)/N$, though instead of just using irrationality of a single ratio $\log 3/\log 2$, one needs some joint incommensurability assertions about multiple logarithms, which the vanilla version of Baker's theorem is not particularly good at getting strong bounds on (but perhaps there are some other results in transcendence theory that could be useful here).



Decomposing a factorial into large factors (second version) | What's new · 3 June 2025

[…] a factorial into large factors". This is a completely rewritten and expanded version of a previous paper of the same name. Thanks to many additional theoretical and numerical contributors from the other coauthors, we now […]



The Guy-Selfridge Conjecture – Kevin Ventullo · 4 June 2025

[…] Rosenfeld, Andrew Sutherland, Terence Tao, and Markus Uhr on a problem posed in this blog post by Tao, which has now culminated in a joint […]



Anonymous · 14 July 2025

Why don't we fit a regression model to the difference between actual log N! and Stirling's approximation, to learn the correction behavior more precisely — maybe even optimize it?